SENIOR ENGINEER ASSESSMENT · TAKE-HOME

Banking Transaction Categorisation

One transaction is posted, placed in one of ten categories by rules an AI tool derived from the supplied history, stored once, and read back; the repository is written as that journey.

Dean Norman PREPARED FOR SOUS

WHAT I BUILT

A categorisation API in plain PHP and MariaDB, with every decision traceable to a prompt or a test

FOR

A reviewer checking the brief: three endpoints, ten categories, Compose, tests, and the AI tool's prompts in the repository.

DONE MEANS

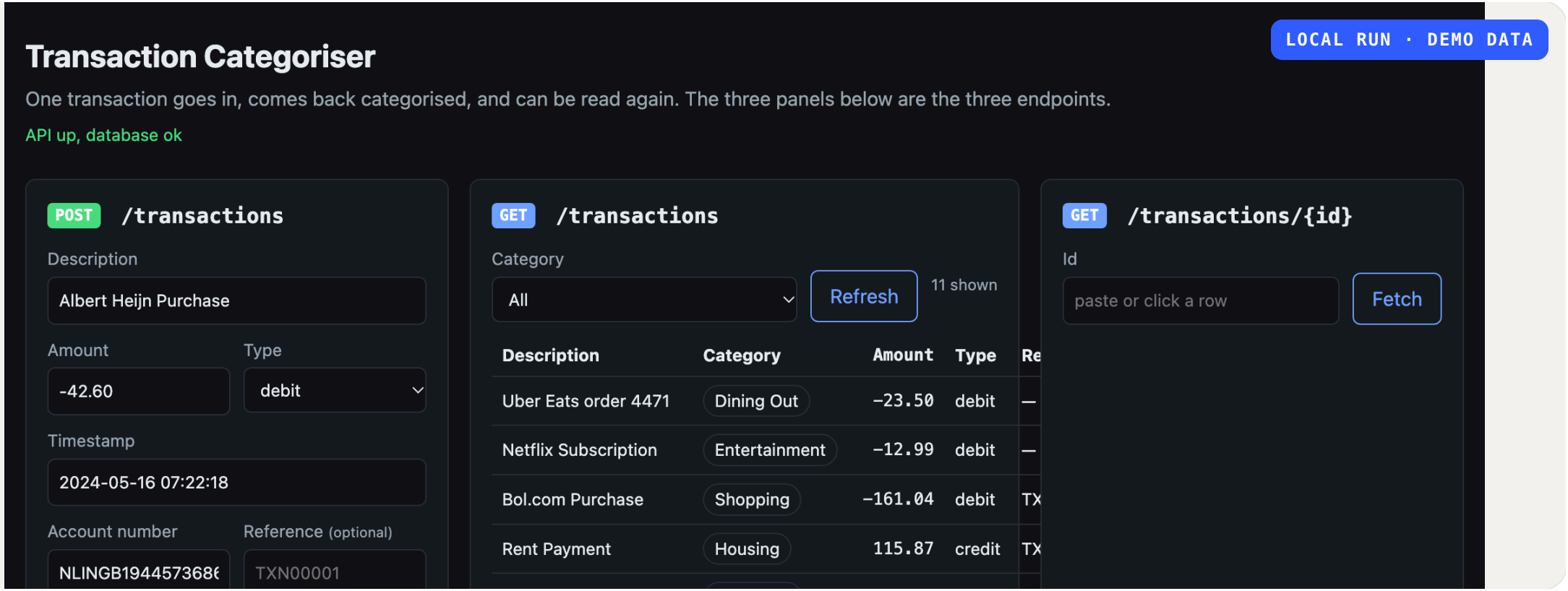
- `POST /transactions` validates, categorises and stores one row: `201`, `422` or `409`.
- Both `GET`s return the shape the POST returned, by id or filtered by category.
- 93 tests inside the stack; 14 smoke requests through nginx.
- Eight steps, eight commits, eight documents.

LEFT OUT

- No CSV import, paging, authentication or runtime LLM call. Each is listed with its next step.

SEE IT

Submit a row from the history and the page shows the category, the rule that decided it, and the raw JSON



HOW IT WORKS

Four calls in order: validate, categorise, insert, answer

01

CODE

`TransactionValidator` checks every field and reports every failure at once as a `422` field map. Money becomes a decimal string, never a float.

02

CODE

`RuleBasedCategoriser` walks 21 ordered, word-bounded keyword rules; first match wins; no match is Miscellaneous with the source `fallback`.

03

CODE

`TransactionRepository` mints a UUID and writes transaction and category in one `INSERT`; the unique key on the bank's reference turns a repeat into `409`.

04

PERSON

The reviewer reads `categorisation.source` on every row: the rule id, or `fallback`, so a wrong category is one line in `rules.php`.

1 insert per transaction, one rule per category decision, no network calls at runtime

DECISIONS

The AI tool derived the rules once; the runtime is deterministic and free

DECISION	WHY	WHAT I GAVE UP
Rules derived by Claude Fable 5.1 at build time, prompts committed	20 distinct descriptions in 1000 rows; a per-request LLM call adds cost, a secret and non-determinism for nothing	Unseen descriptions land in Miscellaneous; the `Categoriser` interface is where an LLM would go
Server-minted UUID, bank id kept as a unique `reference`	Our URL space, idempotency from the reference, reference-less rows still accepted	A client that omits the reference can store a row twice
Amount and type stored exactly as submitted	The history disagrees with itself about direction in half its rows; repairing money values is worse than keeping them	No derived direction field yet
Plain PHP, one front controller, no framework	The stack I ship daily; the router is forty lines and every decision is visible	Middleware, auth and OpenAPI would be hand-written

WHAT I MEASURED

Every description in the supplied history lands where the prompt said it would

93

tests pass, unit and integration, inside the stack

20/20

descriptions pinned to a category and a rule

1000

fixture rows categorise to the expected distribution

14

requests through nginx in the smoke script, all expected statuses

```
docker compose run --rm php vendor/bin/phpunit     OK (93 tests, 334 assertions)
scripts/smoke.sh                                    smoke passed
```

HOW PRODUCTION-READY IT IS

Correct and safe for one client on one box; scale and operations are the listed next steps

DONE

- DONE Every input validated; all failures reported at once; unknown fields ignored
- DONE Duplicate submissions refused by the database, not by application code
- DONE A database outage is a `503` from `/health` and a JSON `500` from a write, never a blank page
- DONE Integration tests on a second database; the suite refuses to run against application data
- DONE List capped at 500 and says so in `meta.cap`

LEFT

- LEFT Paging on the list
- LEFT Authentication and rate limiting as kernel middleware
- LEFT A migrations directory instead of first-start init SQL
- LEFT Re-categorising stored rows when the rule table changes

The rules know twenty descriptions; a bank that writes them differently will reach the fallback

- A description outside the twenty, or spelled without the spaces the patterns expect (`NSInternational`), is Miscellaneous until someone adds a rule.
- Idempotency depends on the client sending `reference`; without it, nothing stops a second row.
- Timestamps without a zone are taken as UTC; the bank's convention was not stated.
- The sign of `amount` and the `type` column disagree in half the supplied rows; both are stored as given and neither drives the category.
- PHPUnit ships in the production image; it should move to a test build target before this is deployed.

NEXT STEPS

Next: paging, then re-categorisation, then an LLM behind the fallback

01 Cursor paging on `GET /transactions` over `(created_at, id)`.

02 A `categorisations` history table and a job that re-runs the rule table over stored rows.

03 An LLM-backed `Categoriser` consulted only when the rules fall back, its answers cached as rule candidates for review.

Run it: `docker compose up -d --build` · 93 tests pass · github.com/DeanNorman/transaction-categoriser-php